

Deleted-Record Recovery from SQLite Databases:

Storage Conventions, a `forbid(unsafe)` Reference Implementation,
and a Multi-Oracle Forensic Evaluation

Albert Kwun Tai Hui ¹

¹Security Ronin

June 2026

Abstract

SQLite is the most widely deployed database engine in the world [1], and the deleted rows inside its files are routinely the evidence an investigator needs—yet the live query interface (`sqlite3`) cannot see them. This paper gives an end-to-end account of where deleted data persists in a SQLite file and how to recover it defensibly. We set out the storage model and the conventions that govern residue lifetime: the comparative retirement of the rollback `-journal` in application contexts in favour of the write-ahead log (`-wal`, which retains *multiple* uncheck-pointed commits), and the destructive effect of `VACUUM` and `PRAGMA secure_delete`. We then describe a complete set of recovery mechanisms—freelist and free-block carving, exact-tile free-block reconstruction, overflow-chain reassembly, dropped-table schema recovery, a WAL commit-history overlay, and rollback-journal before-image folding—unified under a structural *never re-read a live row* (false-positive-zero) guarantee enforced by construction rather than by heuristic.

We contribute: (1) a systematic treatment of SQLite deletion and free-space conventions, including an application case study of 微信 (WeChat) message deletion; (2) `sqlite4n6`, an open, `forbid(unsafe)` Rust reference implementation; and (3) a multi-oracle evaluation—accuracy, false-positive rate, and throughput—against four independent recovery tools (`undark`, `fqlite`, `bring2lite`, and the SQLite Deleted-Records Parser) on independent third-party corpora with published ground truth. On the in-page deletion category of the Nemetz standardized corpus our recall leads the strongest competitor (0.833 vs. 0.798) at precision 1.000 with zero live-row re-reads; across the whole evaluated corpus we re-surface *no* still-live row. Every figure is reproducible from the cited artifacts.

1 Introduction

SQLite ships inside every major browser, mobile operating system, and messaging application, making a SQLite file among the most common artifacts in a digital investigation [1]. The records a custodian *deleted*—a cleared chat, a removed history entry, a purged transaction—are frequently the records that matter, and they often still reside, intact or partially intact, in the file’s free space, in an uncheckpointed write-ahead log, or in a rollback journal. The ordinary access path cannot reach them: `sqlite3` and the language bindings traverse the *live* b-tree and stop, by design.

Recovering that residue is a mature subfield. Nemetz *et al.* published a standardized corpus and showed that no single tool reliably handles all SQLite corner cases [2]; Pawlaszczyk and Hummert [3] and Meng and Baier [4] contributed carving tools and techniques; and the recent survey of Lee, Park, Lee and Choi [5] organizes the field into three families of technique—metadata-based, carving-based, and WAL-based recovery—and evaluates representative open-source tools on recovery rate, reliability, and false positives. Two gaps motivate this work. First,

the literature treats the storage *conventions* that decide whether residue exists at all—WAL versus rollback journaling, `VACUUM`, and `secure_delete`—unevenly, even though they dominate real-case outcomes. Second, recovery tools differ sharply in whether they ever mistake a *live* row for a deleted one, and that false-positive behaviour is rarely guaranteed structurally.

This paper addresses both. We give a precise account of the storage model and the conventions (§2–§3); we describe a full set of recovery mechanisms (§4) under a structural false-positive-zero design (§5); we present an open reference implementation (§6); and we evaluate it against four independent oracles on third-party corpora with published ground truth (§7–§8).

Contributions

1. A systematic account of SQLite’s on-disk deletion behaviour and the free-space lifetime conventions (journal vs. WAL, `VACUUM`, `secure_delete`), with an application case study of WeChat (§2–§3).
2. A structural false-positive-zero recovery design and an open, `forbid(unsafe)` reference implementa-

tion, `sqlite4n6` (§5–§6).

3. A reproducible multi-oracle evaluation—accuracy, false positives, and throughput—on independent corpora with published ground truth (§7–§8).

2 Where Deleted Data Lives

The SQLite database file is a sequence of fixed-size *pages* [6]. A table is a b-tree whose leaf pages hold *cells*, one per row; each cell is a varint-prefixed record of serial-typed values keyed by an integer *rowid*. Recovery hinges on what happens to a cell’s bytes once the row is no longer reachable from the b-tree.

Free blocks and the freelist

When a row is deleted, SQLite does not erase its bytes. The cell is unlinked from the page’s cell-pointer array, and its span is either added to that page’s intra-page *free-block* chain—a linked list of freed gaps inside an otherwise live page—or, when a page empties completely, the page is placed on the database *freelist* of reusable pages [6]. In both cases the original record bytes survive until that space is next *allocated* and overwritten. That window is what every carving technique exploits. A freed cell’s first bytes (the leading payload-length and rowid varints) may be partly overwritten by the free-block header that now occupies the gap; the surviving tail still carries the record’s serial-type array and values.

Overflow pages

A value too large for one page spills onto a chain of *overflow* pages linked by a four-byte next-page pointer [6]. A deleted row with a long `TEXT` or `BLOB` column therefore leaves residue spread across several pages—recoverable only by following, or reconstructing, the chain through the freelist.

The rollback journal and the write-ahead log

SQLite offers two crash-recovery mechanisms, and each leaves a distinct, recoverable side file. The *rollback journal* (`<db>-journal`) holds the *before*-images of pages a transaction is about to modify, so that an aborted transaction can be rolled back. The *write-ahead log* (`<db>-wal`), available since SQLite 3.7.0 (2010) [7], inverts this: new page versions are *appended* to the WAL and only later *checkpointed* back into the main file. The two files are temporal mirror images—the journal holds the pre-transaction state, the WAL the post-transaction states—and both are forensically rich.

A `-wal` holds many commits. A WAL begins with a 32-byte header (a magic number, the page size, and two random *salt* values) followed by *frames*, each a 24-byte frame-header plus one page image.

A frame whose header carries a non-zero “database size in pages” field is a *commit* marker; a single WAL “can and usually does record multiple transactions” [7]. The salts are copied into every frame and re-randomized per transaction so stale frames from a previous journal cannot be mistaken for current ones. Checkpointing is automatic once the WAL reaches a threshold (1000 pages by default) [7]. Until then, the WAL retains the *commit history*: a page version a later commit superseded, or a row a later commit deleted, survives in the earlier frames.

3 Conventions That Govern Residue

Whether deleted data is recoverable at all is decided less by the deletion itself than by three conventions: which journaling mode the application uses, whether `VACUUM` has run, and whether `secure_delete` is enabled.

Rollback journaling vs. WAL

SQLite’s *library* default remains rollback journaling in `DELETE` mode (the journal file is deleted at commit); the other rollback modes are `TRUNCATE`, `PERSIST` (the journal is kept but its header zeroed), `MEMORY`, and `OFF` [8]. Many modern applications, however, *opt into* WAL mode for concurrency, so in practice a `-wal` sidecar is the common case for mobile and desktop application databases while a persistent `-journal` is comparatively legacy. The distinction matters forensically: a `DELETE`-mode journal is removed at commit and leaves nothing on a clean shutdown, whereas a `PERSIST` journal or an uncheckpointed WAL is a durable record of prior state—and the WAL, holding *multiple* commits, is the richer of the two.

VACUUM is anti-forensic

`VACUUM` “rebuilds the database file, repacking it into a minimal amount of disk space” [9]. In doing so it discards the freelist and the free-block slack that carving relies on. The SQLite documentation is explicit: deleted content “can allow deleted content to be recovered ... Running `VACUUM` will clean the database of all traces of deleted content, thus preventing an adversary from recovering deleted content” [9]. A vacuumed database is, for residue purposes, scrubbed. `auto_vacuum` incrementally reclaims emptied pages but “does not compact partially filled pages” [8], so it destroys whole-page freelist residue while leaving intra-page free-block residue.

PRAGMA secure_delete

With `secure_delete=ON`, SQLite overwrites freed content with zeros at deletion time, eliminating residue; it is *off* by default, and a later `FAST` variant zeroes content only when doing so adds no I/O [8]. An investigator must therefore treat the presence of recoverable residue as evidence that `secure_delete` was *not*

in force—and its absence as inconclusive rather than as proof of secure deletion.

Application case study: 微信 (WeChat)

WeChat, the dominant Chinese super-app, illustrates how application conventions shape recovery. On Android, chat history is stored in an SQLCipher-encrypted database `EnMicroMsg.db` (messages in the `message` table, contacts in `rcontact`); on iOS the store is `MM.sqlite`, often unencrypted [10]. The Android database key is derived as the first seven hexadecimal characters of `MD5(IMEI || UIN)` [10]. Hur *et al.* show that deleted WeChat messages can be recovered from the tokenized index of the full-text-search database (`FTS5IndexMicroMsg.db`) even after the message row is gone, and give a decryption method for the IMEI-absent case (Wi-Fi-only tablets); they further note that attachment metadata persists in the `appattach` table after deletion and that deletions are not synchronized across the Android and Windows clients, so a message purged on one may survive on the other [10]. The recovery techniques in this paper apply once such a database is decrypted to plaintext SQLite: the deleted rows live in the same free blocks, overflow chains, and WAL frames as any other SQLite artifact.

4 Recovery Mechanisms

We implement the three technique families of the Choi survey [5]—metadata-based, carving-based, and WAL-based—as the following mechanisms. Each emits a labelled *recovery source* and a confidence (§5); each preserves the live-row invariant by construction.

Free-block and freelist carving (metadata- and carving-based). Whole pages on the freelist contain only deallocated content and are the strongest case; intra-page free blocks are gaps in still-live pages and are more likely to be partly overwritten. We carve both, decoding each candidate cell against the live schema and reading every value in its native serial type.

Exact-tile free-block reconstruction. When a freed cell’s leading bytes are destroyed by the free-block header, the cell length and `rowid` are gone but the serial-type tail survives. We rebuild the record from the surviving tail plus the owning table’s schema template, accepting a reconstruction only when it *exactly tiles* the free block—its decoded extent must end precisely on the block boundary. This exactness gate is what rejects column-shifted phantoms, and it extends to two-byte `rowids` (≥ 128) and to runs of coalesced freed cells, where each cell in the span must tile in turn. The destroyed `rowid` is surfaced as absent rather than guessed.

Overflow-chain reassembly (carving-based). A deleted row whose payload spilled onto overflow pages is reassembled by following the freed chain through the freelist leaves to a byte-exact body. Because a freelist leaf can be stale, this path is graded below an intact in-page cell.

Dropped-table schema recovery (metadata-based). A dropped table’s definition survives as a deleted `sqlite_master` row in page-1 free space. We recover the dropped table, index, view, or trigger name and its `CREATE` statement, which both documents the deletion and supplies a schema template for attributing other residue.

The WAL commit-history overlay (WAL-based). We parse the WAL frame by frame, reconstruct the database state at each `COMMIT` marker, and surface rows that were live at an earlier commit but deleted by a later one. Each recovered version carries its log-sequence provenance (the two salts and the frame index), giving an ordered edit history rather than a flat dump. There is no wall-clock timestamp in a WAL—only this commit order.

Rollback-journal before-image folding (WAL-based, temporal inverse). When a `<db>-journal` is present we fold in the *before*-images of the last transaction: rows it deleted and the pre-edit values of rows it modified. This is the temporal inverse of the WAL—where the WAL holds after-images, the journal holds the before-image of the last transaction.

Tier-2 fragment salvage. Where a full row cannot be reconstructed but a *distinctive* cell survives at a structural anchor (a `TEXT` value of at least four UTF-8 bytes, or a `REAL`), we salvage the maximal decodable column prefix as a *fragment*—a typed object with no `rowid` and an incomplete column set, kept in a separate output surface so it can never be mistaken for a full recovered row.

5 Never Re-Reading a Live Row

In a forensic setting, reporting a still-*live* row as “deleted” is the costliest error: it fabricates evidence. Our design makes that outcome structurally impossible, in two layers.

Structural exclusion. We first compute the byte extents of every *live* cell directly from the b-tree structure. The set of carvable free regions is then defined as the *complement* of those extents within each page. Carving runs only inside free regions, so a row still reachable from the live b-tree is never scanned for recovery in the first place.

Value-collision filter. A freed cell’s bytes can, in principle, coincide with a live row’s values (for example, a freed copy left by an in-place `UPDATE`). After

Table 1: Confidence assigned by recovery path.

Conf.	Recovery path
0.9	full row, distinctive text (freelist / WAL frame / live-anchor)
0.72	in-page free block, distinctive text (0.9×0.8)
0.675	overflow-chain reassembly (0.9×0.75)
0.6	full row, no distinctive text
0.48	in-page free block, no distinctive text (0.6×0.8)
0.4	exact-tile free-block reconstruction
0.3	free-block reconstruction spilling to overflow (0.4×0.75)
0.2	Tier-2 fragment

carving, any recovered record whose decoded values equal those of a live row in the same table is dropped. What remains—an UPDATE’s freed *prior* version, whose values differ from the live row—is genuine deleted edit history, not a duplicate.

These two layers are independent of confidence: a still-live row is excluded regardless of any threshold. Confidence, by contrast, manages *phantoms* (coincidental byte sequences that decode to a plausible but spurious record), and is reported, never used to suppress a live-row collision.

Confidence model. Every recovered item carries a confidence set by its recovery path (Table 1). A base of 0.9 for a full row with distinctive text is attenuated by an in-page factor (0.8) for free-block residue and by an overflow-chain factor (0.75) for chain-reassembled bodies; freeblock reconstructions take a fixed 0.4 and fragments 0.2. The CLI exposes a `--min-confidence` ladder (`info 0.0`, `low 0.2`, `medium 0.4`, `high 0.6`, `critical 0.8`) that filters every emitted item; it is a recall/noise dial, not a safety control.

6 Reference Implementation

`sqlite4n6` is an open-source Rust workspace of three crates: `sqlite-core` (a panic-free, read-only file-format reader: b-tree, WAL, freelist, overflow), `sqlite-forensic` (the carving and audit harness with the recovery oracles), and `sqlite4n6` (the command-line tool). All three are `forbid(unsafe)`; the project pins a current Rust toolchain, declares a conservative minimum supported version, and gates on 100% function coverage in continuous integration. The reader opens evidence *read-only* and, for WAL artifacts, in immutable mode, so analysis never mutates or checkpoints the file under examination.

Output is designed for review rather than for reparsing. The default carve writes a per-rowid *version-history* workbook—live rows interleaved with the prior and deleted versions recovered from the WAL, the roll-back journal, and free space, ordered by commit se-

quence and tinted by state—with image BLOBs rendered as in-cell thumbnails. A `-f db` option emits a queryable SQLite database whose recovered rows are attributed in three honest tiers (observed fact, forensic inference, unknown), and `-f jsonl/csv` stream the raw records. Every record carries provenance columns: page, byte offset, `rowid` (or absent), recovery source, and confidence.

7 Validation Corpus

A recovery tool validated only against fixtures its own authors built inherits their blind spots. We therefore validate primarily against third-party corpora whose *ground truth was authored by someone else*, and label every result by the strength of the check that vouches for it (Table 2).

The head-to-head accuracy results below use the Nemetz standardized corpus [2], whose authors published a per-row answer key for every deleted record. We score the three deletion categories with a format-stable identity (in-page deletion 0C, deleted-then-overwritten 0D, and deleted overflow 0E); categories whose only key column is a floating-point value rendered differently by each tool are excluded from the cross-tool key, as are the whole-table-loss and anti-forensic categories that have no deleted-row anchor.

8 Evaluation

Accuracy: a multi-oracle head-to-head

We compare `sqlite4n6` against four independent recovery tools—`undark` (C) [13], `fqlite` (Java) [3], `bring2lite` (Python) [4], and the SQLite Deleted-Records Parser (SQL-DRP, Python) [14]—each scored against the *same* Nemetz answer key with the same two-column identity matcher (Table 3). Recall is computed on the substrate of rows whose identity still survives in the artifact; precision counts phantom rows; the “live” column counts still-live rows wrongly reported as deleted. `fqlite` is driven through a headless source-instrumentation tap of its carving engine; the two Python tools through normalizing wrappers. The patches, shims, and exact versions are committed so the comparison is independently reproducible.

On in-page deletion (0C) our carver recovers 70 of 84 recoverable rows (recall 0.833) ahead of `fqlite`’s 67 (0.798), and does so at precision 1.000—zero phantom rows and zero live re-reads—against `fqlite`’s 0.798. On deleted-then-overwritten rows (0D) our carver and `fqlite` both recover every row whose full identity survives, at precision 1.000; on the small overflow substrate (0E) our carver recovers all four at precision 1.000, including one reassembled overflow chain. Across the evaluated corpus our carver re-surfaces *no* still-live row; `undark`, which dumps live and deleted

Table 2: Evidence tiers. Tier 1 = an independent party authored both the artifact and the answer key (or it is real-device data); Tier 2 = real `sqlite3`-engine bytes whose ground truth is derivable from the documented construction or confirmed by an independent oracle; Tier 3 = authored by us with nothing independent vouching.

Tier	Corpus / oracle	What it establishes
1	Nemetz standardized corpus [2] (141 DBs, CC0, per-row <code>.xml</code> keys)	deleted-record recall & precision vs. third-party ground truth
1	NIST CFReDS / CFTT SQLite sets [11] (SFT-01/03/05)	text encodings; WAL & journal recovery; native types + BLOBs
1	NIST CFReDS <i>Data Leakage</i> (<code>snapshot.db</code> from a Volume Shadow Copy)	real-case end-to-end recovery of two deleted rows, one freeblock-clobbered
1	DC3 <code>sqlite_dissect</code> corpus [12] (no-deletion baseline)	zero false positives on unallocated content
1	<code>sqlite-unhide</code> corpus (9 DBs, independent <code>.txt</code> keys)	two-byte-rowid free-block recovery; surfaced two real defects
1	Josh Hickman iOS-17 images (genuine app databases)	no-panic robustness over real device data
2	real- <code>sqlite3</code> fixtures; <code>sqlite3 .recover</code> ; <code>calamine</code> XLSX reader	construction-derivable answers; differential agreement; output round-trip

Table 3: Deleted-record recovery on the Nemetz corpus (categories OC/OD/OE; D_{rec} = recoverable-substrate denominator). All recall/TP figures are harness-computed against the published answer key. `fqlite`’s precision is measured through our committed headless tap.

Cat	Tool	TP	FP	FN	Rec.	Prec.
OC	ours	70	0	14	0.833	1.000
	fqlite	67	17	17	0.798	0.798
	bring2lite	40	0	44	0.476	1.000
	undark	14	10	70	0.167	0.583
	sql-drp	0	1	84	0.000	0.000
OD	ours	19	0	0	1.000	1.000
	fqlite	20	0	0	1.000	1.000
	bring2lite	7	0	12	0.368	1.000
	undark	1	10	18	0.053	0.091
	sql-drp	0	0	19	0.000	—
OE	ours	4	0	0	1.000	1.000
	undark	3	6	1	0.750	0.333
	bring2lite	3	5	1	0.750	0.375
	fqlite	2	6	2	0.500	0.250
	sql-drp	0	0	4	0.000	—

rows together, re-reads 56 live rows as deleted on OD alone. Figure 1 plots the precision–recall plane.

False positives across forensic hazards

Accuracy on a clean corpus says little about the hazard that matters most—mistaking live data for deleted. We replicate the three adversarial scenarios of the Choi survey [5] on real `sqlite3`-engine bytes (a Tier-2 *replication* of the published construction, not the authors’ own files): 0F, where a b-tree rebalance moves a live row onto a freed page; 0B, a drop-and-recreate of a same-schema table; and 10, a `secure_delete` wipe of the main image leaving residue only in the WAL. On 0F, `sqlite4n6` recovers 33 of 50 with *zero* live false positives, where `bring2lite` re-surfaces 13 live rows;

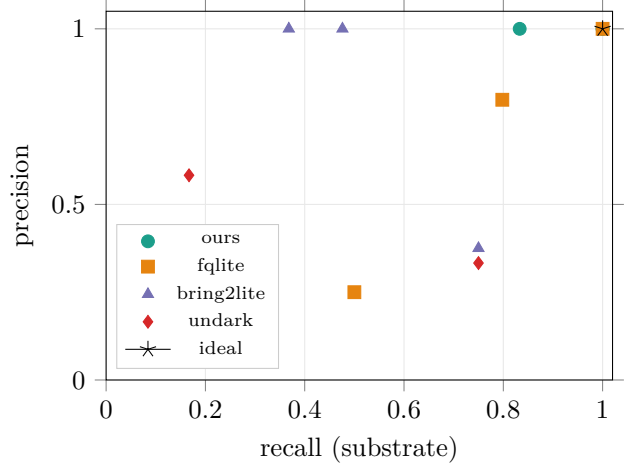


Figure 1: Precision–recall across categories OC/OD/OE (SQL-DRP omitted: it recovers no answer-key row under the format-stable identity). Our carver sits on the precision-1.0 ceiling in every category.

on 10, the WAL-only scenario, both WAL-aware tools recover all 20, while the main-image-only tools recover none.

Throughput

On a generated 100 MB database with 80,000 deleted rows (single machine, median of five runs), `sqlite4n6` streaming JSONL recovers 79,904 deleted rows (recall 0.9988) with zero live false positives in 15.6 s. The faster tools do different work: `undark` dumps 177,906 rows—live and deleted together, undifferentiated—in 1.5 s, and SQL-DRP emits 5 printable-string carves in 0.2 s. Throughput is therefore not directly comparable across tools that recover different things; we report it for completeness, not as a headline.

9 Related Work

The Choi survey [5] is the closest prior work: it organizes SQLite deleted-record recovery into metadata-based, carving-based, and WAL-based families and evaluates the same open-source tools we compare against, with explicit attention to false positives. We extend it with a structural false-positive-zero *guarantee* and a reproducible head-to-head on the standardized corpus. Nemetz *et al.* [2] contributed the standardized corpus and the finding that no tool handles every corner case—which our multi-oracle evaluation re-confirms tool by tool. **fqlite** [3] and **bring2lite** [4] are the strongest open carvers and recover free-block and freelist residue; **undark** [13] dumps all intact records without separating live from deleted; SQL-DRP [14] carves printable strings from freed regions; and DC3’s **sqlite_dissect** [12] parses the database together with its journal and WAL. On the application side, Hur *et al.* [10] recover deleted WeChat messages from full-text-search index residue, complementary to the free-space and WAL recovery treated here.

10 Discussion and Limitations

Recovery is bounded by the conventions of §3: **VACUUM** and **secure_delete** destroy residue, and an allocation that overwrites a freed cell ends its recoverability. Our end-to-end recall on overflow bodies is low (0.333 on OE) precisely because most deleted overflow payloads in the corpus were genuinely overwritten—an honest ceiling, not a defect, and one our substrate recall of 1.000 makes explicit. The competitor precision figures we report are measured through our own headless taps and wrappers; recall, the substantive engine result, reproduces exactly, while a tool’s phantom count depends on its emission policy, which we re-derived for **fqlite**. Finally, our false-positive scenarios are a replication of the Choi survey’s construction on real-engine bytes rather than the authors’ own files; we label them Tier-2 accordingly.

11 Conclusion

Deleted SQLite records persist in free blocks, overflow chains, rollback journals, and—most richly—in the multiple uncheckpointed commits of a write-ahead log, until **VACUUM**, **secure_delete**, or reallocation erases them. We have described a complete set of recovery mechanisms for that residue, unified by a structural guarantee that a still-live row is never reported as deleted, and an open **forbid(unsafe)** reference implementation. Measured against four independent tools on third-party corpora with published ground truth, the implementation leads on in-page recall at precision 1.000 and re-surfaces no live row across the evaluated corpus. Every number is reproducible from the cited artifacts.

References

- [1] SQLite Consortium. Most widely deployed and used database engine. <https://www.sqlite.org/mostdeployed.html>, 2026. Accessed 2026-06.
- [2] Sebastian Nemetz, Sven Schmitt, and Felix C. Freiling. A standardized corpus for SQLite database forensics. *Digital Investigation*, 24:S121–S130, 2018. doi: 10.1016/j.diin.2018.01.015. Proceedings of DFRWS EU 2018.
- [3] Dirk Pawlaszczyk and Christian Hummert. Making the invisible visible—techniques for recovering deleted SQLite data records. *International Journal of Cyber Forensics and Advanced Threat Investigations*, 1(1–3):27–41, 2021. doi: 10.46386/ijcfati.v1i1-3.17.
- [4] Christian Meng and Harald Baier. bring2lite: A structural concept and tool for forensic data analysis and recovery of deleted SQLite records. *Digital Investigation*, 29:S31–S41, 2019. doi: 10.1016/j.diin.2019.04.017. Proceedings of DFRWS USA 2019.
- [5] Seonghyeon Lee, Sooyoung Park, Insoo Lee, and Jongmoo Choi. A comprehensive analysis and evaluation of SQLite deleted record recovery techniques: A survey. *Forensic Science International: Digital Investigation*, 55:302031, 2025. doi: 10.1016/j.fsidi.2025.302031.
- [6] SQLite Consortium. The SQLite database file format. <https://www.sqlite.org/fileformat2.html>, 2026. Accessed 2026-06.
- [7] SQLite Consortium. Write-ahead logging. <https://www.sqlite.org/wal.html>, 2026. Accessed 2026-06; WAL available since 3.7.0 (2010-07-21).
- [8] SQLite Consortium. Pragma statements supported by SQLite. <https://www.sqlite.org/pragma.html>, 2026. Accessed 2026-06; see **secure_delete**, **auto_vacuum**, **journal_mode**.
- [9] SQLite Consortium. **VACUUM**. https://www.sqlite.org/lang_vacuum.html, 2026. Accessed 2026-06.
- [10] Uk Hur, Myungseo Park, and Jongsung Kim. Study on improved decryption method of WeChat messenger and deleted message recovery using SQLite full text search data. *Journal of the Korea Institute of Information Security and Cryptology*, 30(3):405–415, 2020. doi: 10.13089/JKIISC.2020.30.3.405.
- [11] National Institute of Standards and Technology. Computer forensic reference data sets (cfreds): SQLite data recovery. <https://cfreds.nist.gov/>, 2021. CFTT SQLite recovery read-me <https://www.nist.gov/document/cftt-document-sqlite-recovery-read-me>; accessed 2026-06.

- [12] DoD Cyber Crime Center (DC3). SQLite dissect. <https://github.com/dod-cyber-crime-center/sqlite-dissect>, 2026. Accessed 2026-06.
- [13] Paul L. Daniels. undark: an SQLite deleted and corrupted data-recovery tool. <https://pldaniels.com/undark/>, 2026. Code mirror <https://github.com/inflex/undark>; accessed 2026-06.
- [14] Mari DeGrazia. SQLite deleted records parser. <https://github.com/mdegrazia/SQLite-Deleted-Records-Parser>, 2026. Accessed 2026-06.